



IX CAIM
IV CAIFE



FIE
Facultad de
Ingeniería
del Ejército



FoDAMI

17, 18 y 19 de septiembre 2025

MBSE-SUPPORTED MECHATRONIC DESIGN OF A MOBILE ROBOT FOR INDOOR MATERIAL TRANSPORTATION IN A METALWORKING FACTORY

Guillermo Rolle^{1,2}, Matías Nacusse¹, and Sergio Junco¹

¹Laboratorio de Automatización y Control (LAC), EIE-FCEIA-UNR; Riobamba 245bis, 2000 Rosario, Argentina

²CONICET

*Email: {grolle,nacusse,sjunco}@fceia.unr.edu.ar

ABSTRACT

This paper presents the *ongoing* development of a mobile robot prototype (“the System”) intended for the autonomous movement of materials, products, and by-products between the warehouse and internal production stations of a factory, ensuring the safety of operators and other human beings, the integrity of the materials transported, and operational efficiency.

The structured application of modeling for system requirements, analysis, design, verification, and validation, as specified by the ARCADIA MBSE (Model-Based Systems Engineering) methodology, is detailed alongside the introduction of each development phase of the project. Their correlations with established mechatronic design methodologies are outlined.

Accordingly, the following succession of “perspectives” is performed: **Operational analysis** (what System users must achieve); **System needs analysis**: (what the System must achieve for users); **Logical architecture** (how the System will work to meet expected aims); **Physical architecture** (how the System will be built); This process first identifies Interacting Entities and Actors (warehouse, materials, production stations, production operators), Capabilities (materials, products and by-products must be transported between the warehouse and the production stations; supplies must be replenished before running out at each station) and Activities (to be performed at the Entities and/or by the Actors). This is followed by finding the functions of System and Actors (loading/unloading supplies, navigating the factory, transporting supplies, processing transport requests) and defining the functional chains for the transport of materials and products as well as those of material and product requests. The subsequent phase involves the design of the solution architecture, first at logical, then at physical level. Main architectural components are mobile base (including locomotion system and actuators); navigation system (including sensors and controllers); suspension system; energy system; operator’s communication interface. The article details the formalisms, mathematical modeling methods, and tools utilized to achieve the anticipated solution.

Keywords: *Model-Based Systems Engineering; Arcadia/Capella; Mechatronic Design; V-method; Mobile Robots*



17, 18 y 19 de septiembre 2025

1. INTRODUCTION

In all production factories there is a flow of inputs, materials, products, by-products, etc., among warehouses, working stations, repair shops, etc. The efficient transportation of all these items is a key element regarding productivity: if a worker is not provided timely with the inputs needed to perform the tasks at his/her station, a production delay is unavoidable, reducing overall productivity.

The case study presented here addresses a small factory's desire to free operators from their current task of searching for and transporting materials to and from their workstations. This fact clearly generates production delays, degrading the company's overall efficiency. To increase efficiency and competitiveness, the owner aims at obtaining a solution that simultaneously focuses on simplicity, low cost and upgradability. The latter feature is a key requirement, as due to its limited financial capacity, the company is constrained to first build and put in service a basic version of the solution, which should be later gradually enhanced with further functionalities.

The overall goal of this work is to provide a robotic solution addressing the client needs, with particular attention to the upgradeability requirement. A key issue in this sense is that construction upgrades should have already been considered in the development stages, so that large unexpected modifications shall not be required to be implemented when upgrading the current system. This is only possible if the whole development problem is tackled with a comprehensive mechatronic design approach, using formal, mathematical and simulation models of the system under development, including its future construction versions [1]. This calls for the use of MBSE methodologies to define and design the robot.

The mechatronic design concept is characterized by the simultaneous consideration of all engineering aspects from the very start of the design process, enabling the creation of novel, more efficient, and high-quality designs, with reduced costs and faster time to market. However, this integrated approach introduces inherent complexity into the system design process [2]. This demands systematic procedures employing methodologies that precisely define the initial requirements, which subsequently evolve into more complex requirements, guiding all stages of the design process.

All these prerequisites placed on the mechatronic design call for a methodology which is clearly fulfilled by the Model-Based Systems Engineering (MBSE) approach. Indeed, MBSE focuses on the creation of an integrated, coherent, and consistent 'System Model' that serves as the source of truth, enabling the development and inter-connection of multiple views throughout the entire system lifecycle [3]. In this approach, models serve not only as tools for documenting engineering outcomes but also as foundational instruments for simulating and analyzing potential solutions. Among the most well-known MBSE-methodologies are: IBM Telelogic Harmony-SE, INCOSE Object-Oriented Engineering Method (OOSEM), IBM Rational Unified Process for Systems Engineering (RUP SE), Vitech MBSE Methodology, JPL State Analysis, Dori Object Process Methodology (OPM) [4], the more recently released ARCADIA [5], Magic Grid [6] and Integrated Systems Engineering and Pipelines of Processes in Object-Oriented Architectures (ISE&PPOOA) [7]. Readers could refer to [8] and [9] for a sound review of MBSE approaches and methodologies.

The V-model of mechatronic design proceeds recursively going through the Requirements, Functional, Logical, Physical (RFLP) stages in the descendent branch of the V, and the Integration, Verification and Validation phases in its ascendent branch. These stages ensure that requirements are captured through functions, logical elements,



17, 18 y 19 de septiembre 2025

and physical components, and are integrated into a consistent system model [10]. The V-model has been used to MBSE-supported design of mechatronic systems, in combination with the VDI guideline 2206 [11]. In particular, in [12] a preliminary design of a mechatronic interface between Mobile Robots and Cobots is presented. [9] presents a design workflow for safety-critical mechatronic systems applied to a winged Unmanned Aerial Vehicle. The workflow relies on the V-model for mechatronic systems development and the RFLP process for the descending side of system design.

In [13] the authors compare MBSE and traditional approaches (non-MBSE) in the architectural design of a space robotic system, focusing on the evaluation of automatic information transfer. Their study quantitatively demonstrates that the MBSE approach contributes to a significant reduction in design errors and development costs by enabling seamless and automated data exchange between subsystems. Moreover, MBSE enhances traceability throughout the project lifecycle, leading to greater consistency, better requirements management, and improved integration across engineering domains.

These advantages are particularly valuable in the context of complex and high-risk space missions. In [7], the ISE&PPOOA methodology for MBSE is applied to the development of collaborative robotic systems. In [14], a conceptual design of a Delta robot is developed using black-box/white-box analysis and presented with SysML as the formal modeling tool. Additionally, in [15], the modeling and design of a nuclear inspection robot were carried out using the CESAMES system architecture method (CESAM). Finally, in [16], a reference model using SysML was employed in the safety design of a modular service robot for a stocking robot application.

In this paper, we present the current state of the ongoing robot design following ARCADIA as a method that guides the implementation of MBSE to capture the required information in the system descriptive models, and as a modeling language to capture the information in a descriptive system. For creating and managing models, its associated tool, Capella [17], is used. Specifically, the following advances are presented (along with their methodological derivation): the precisely identified client's needs and the rigorous definition of the requirements; the robotic functional solution and the logical structure of its mobile base; the system's control law; the design of the main physical architecture, including its mechanical components and actuators; the constraints that will drive each of the domain-specific design activities.

The paper is organized as follows. Section 2 –Language, Method and Tool– presents the elements of Arcadia/Capella relevant to this work in conjunction with the use of the V-model as a design guide. Section 3 –System Design– deals with the application of the methodology to the specific problem tackled. Section 4 –Detailed Design– describes how a first rough estimation of the main parameters/properties that will drive the domain-specific design processes are obtained. Most relevant for the desired upgradability of the basic platform, these properties are traced to system or stakeholder requirements which were consistently refined throughout the design process. Finally, Section 5 presents the conclusions.

2. LANGUAGE, METHOD & TOOL

MBSE relies on three pillars: a standardized *modeling language* to capture the information in a descriptive system model, a *method* that guides the implementation of MBSE to capture the required information in the system descriptive models and, a *tool* to create and manage models [18].



IX CAIM
IV CAIFE



FIE
Facultad de
Ingeniería
del Ejército



FoDAMI

17, 18 y 19 de septiembre 2025

According to [19], mechatronics is the *synergetic integration* of mechanical engineering, electronic engineering, informatics and control engineering. A design methodology for mechatronic systems [11] adapts the V-Model from Software Engineering to Mechatronic Systems Engineering. It consists of three parallel strands shaping a V. Figure 1 presents it integrated with ARCADIA (short for ARChitecture Analysis and Design Integrated Approach, details below): the central strand represents core design activities, which are accompanied by *management of requirements* (inner strand) and *modeling & analysis* activities (outer strand) throughout the whole design process. The descending branch (left side) of the V-model represents the core activities of systems architecturing (i.e. *requirements elicitation* and *system architecture & design*). Next, at the bottom (middle part), domain-specific design activities take place. Lastly, at the ascending branch (right side), system *integration, verification* and *validation* (IVV) activities are performed to ensure that the system satisfy the requirements and needs. As explained in [11], during the *requirement elicitation* activity, stakeholder needs are used to derive and formalize requirements. Initially, these requirements reflect the stakeholder perspective and do not describe the system itself. They are later converted into system requirements that define the system to be developed, resulting in the system specification: the required system characteristics and context. During the *system architecturing & design* activity an overall solution structure of the system is developed based on the functional structure and the underlying principles of action. The best possible allocation of the individual functions to the disciplines involved is determined, and the system is decomposed into implementable units which, in turn, define the input constraints for the detailed design activities.

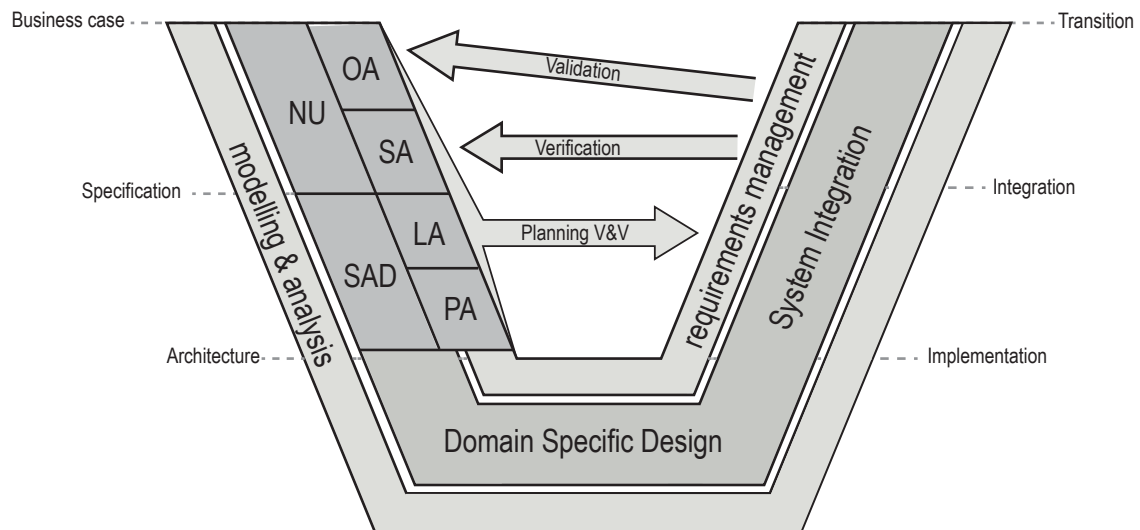


Figure 1: Integration of the V-Model for mechatronic design with the ARCADIA method

On the other hand, Arcadia (short for ARChitecture Analysis and Design Integrated Approach) is both a MBSE **LANGUAGE** and **METHOD** [5]. It focuses on the design of systems architectures and is supported by a dedicated **TOOL**, Capella [17], that responds to the constraints involved with full-scale application in an operational context.



IX CAIM
IV CAIFE



FIE
Facultad de
Ingeniería
del Ejército



FoDAMI

17, 18 y 19 de septiembre 2025

The Arcadia method is structured around four main perspectives, each serving a specific purpose in system development: *Operational Analysis* (OA), which focuses on what users need to achieve; *System Need Analysis* (SA), which defines what the system must accomplish for users; *Logical Architecture* (LA), which describes how the system will work to meet expectations; and *Physical Architecture* (PA), which details how the system will be built. Furthermore, OA and SA perspectives are grouped under *Need Understanding* (NU) activity, whereas LA and PA perspective are grouped under *Solution Architectural Design* (SAD) activity.

In comparison to the V-model detailed above, the Arcadia method begins with OA, which focuses the analysis on users without referencing the system, followed by SA, which captures the system needs and context from OA and external requirements, as detailed in [5]. Both perspectives align with the *requirements elicitation* phase of the V-Model, focusing on identifying and organizing system characteristics and requirements. In LA, an internal functional analysis of the system is conducted, describing the functions to be performed and guiding initial design decisions and, finally, in PA, the final system architecture is defined, including implementation details, technical choices, and the behavioral components necessary to execute the system functions. These two perspectives relate to the *system architecture & design* activity of the V-Model. Lastly, Arcadia supports the planning of Integration, Verification, and Validation (IVV) activities by defining scenarios and functional chains at all design levels.

3. SYSTEM DESIGN

In this section the thinking process for the development of the mechatronic system following the ARCADIA method is detailed: firstly, the Operational Requirements are formalized from the stakeholders needs and the System Specification is derived. Afterwards, the Solution Architectural Design is described at the Logical and Physical Architecture perspectives.

3.1 Operational Analysis

The stakeholder's primary objective is to avoid delays due to the unavailability of production goods, which results in the need to improve or even rethink the goods transportation system. This is further refined in two Operational Capabilities: the obvious expectation of goods to be transported, and the other captures the goal of having goods replenished before they run out. Operational Entities/Actors and Operational Activities are identified: *Warehouse*, as the place where materials and products are stored; *Working Stations*, as they make productive inputs available for manufacturing and store their local production; and *Production Workers* as they use productive goods to manufacture products or by-products.

3.2 System Need Analysis

The system's mission is to *transport productive goods autonomously within the factory*. To accomplish its mission, the system shall be able to navigate autonomously, handle production goods, and transport goods safely. The system shall allow workers to request the transportation of goods on demand in order to guarantee the on-time supply of goods. At a first product maturity level, workers are responsible for verifying the availability of goods and requesting new supplies before they run out, but, in a future version, an additional capability for autonomous



detection of the level of supplies could be added. Lastly, risk detection and avoidance capability is also required to guarantee safety and prevent accidents with workers. Extra non-functional requirements are incorporated: low cost and design simplicity, maximum load specification and details on some particular goods to be handled, navigation should happen inside the circulation paths painted on the floor, etc.

In Figure 2, the system architecture and context is shown. The System is assigned functions for navigation (*navigate*, *plan trajectory*, *self-localise*), goods handling (*load/unload goods*) and transportation (*transport goods*). A function to *process transport requests* is allocated to the system which computes the transport cycle for an incoming request. The functional chains representing *materials transportation* and *materials request* processes are visualized (similar representation for *products transportation* not shown in order to save space).

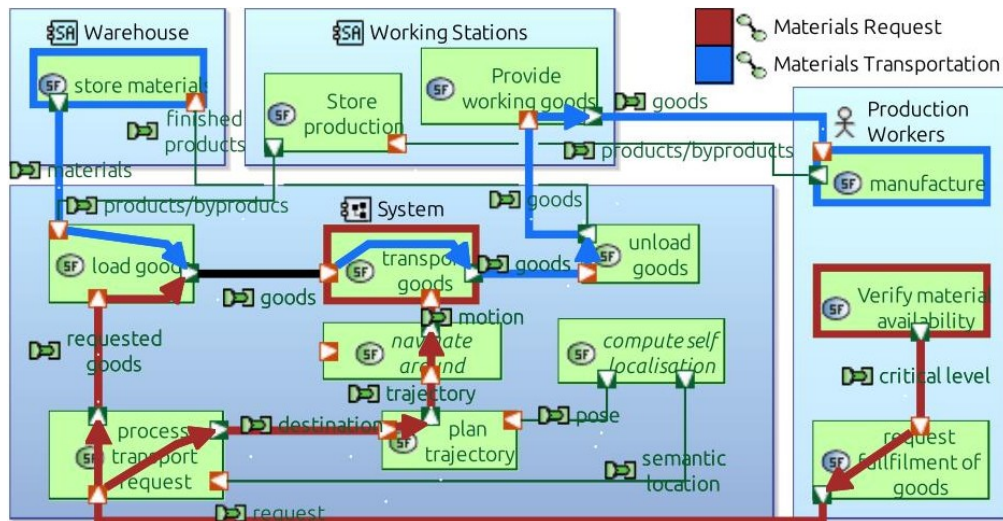


Figure 2: System Achitecture Blank: Main System Functions and Functional Chains

Details on self-localisation. The outputs of the *self-localise* functions are *pose* and *semantic location*. *pose* is both position and orientation of the mobile base. *semantic location* represents higher-level locations within the factory: robot base, warehouse, working station 1 (WS1), WS2, WS3, etc. Semantic locations are more helpful for defining transport request cycles than pose coordinates.

Details on transport requests. At a particular working station, several input goods may be required: some of them may be provided directly from the warehouse but some others might be by-products from another working station. When workers request the fulfillment of goods, three exchange items are required: “what” kind of material is requested, “from” which location are goods to be transported from and “to” which location are goods to be transported to. Then, the transport cycle is defined as: go to “from” → load “what” → go to “to” → unload “what” → if pending requests: repeat process; else, go to robot base and wait.

3.3 Logical Architecture

When designing the logical structure of the system, it's rather straightforward to decompose it into two main components: the *mobile base* and the *operation structure*. The former is responsible of executing all the mobility-related functionalities (navigation, trajectory planning, transportation, etc.), while the latter is responsible for the material handling functions (loading/unloading and/or other manipulation functions). They could be considered independent subsystems that interact with each other, so the general design process for mechatronic systems could be applied to each of them separately, importing the requirements acquired at the upper-system architectural design considering each of them as external actor to one another. For this reason, we will focus only on the design of the mobile base, highlighting also the modular aspects of the solution (as required by the stakeholder): at a first step, the stakeholder could build a simpler version of the final solution consisting only of a mobile robot for transporting materials which are loaded and unloaded by an operator. Later, the operational structure could be developed and added to the mobile system, as long as the mobile base was designed with this requirement in mind.

In the following paragraphs, a logical decomposition of the *mobile base*, as shown in Figure 3, is presented.

One of the main subcomponents is the *mobile cart* which is responsible for allowing the system to move around the environment, whether manually or autonomously. The only valid alternatives considered (due to costs and complexity) are single chassis wheeled robots whose behavior depends on their kinematic configuration [20]. Subsequently, the mobile cart consists of: a *chassis*, that rigidly links wheels and other components, and provides structure and mounting points to the system; a *wheel configuration*, that provides stability, distributes the weight of the whole base and provides and characterises motion behaviour via the specific implementation of *traction wheels* to transfer rolling power, *steerable wheels* that allow to change the orientation of the wheel's plane (modifying the motion constraints) and *unconstrained wheels* to provide support without restricting motion; *actuators configuration*, that transfers power to some of the wheels; and *suspension*, which is required to absorb vibrations and irregularities while also guaranteeing cart's stability.

The *navigation module* is responsible for executing the motion control logic that computes the actuator's commands, the trajectory planning and the self-localisation algorithms. These functions are allocated to subcomponents that specifically implement them in more detail (currently under development, not shown in this work).

The *power module* provides the necessary energy and power to the actuators to functionally work. Lastly, the *interface module* allows intercommunication between the workers and the system. It receives the transport request and commands the navigation module to accomplish the production cycle explained in section 3.2.

Details on Motion Controller

The control algorithm depends both on the kinematic and actuator configurations. Here, a trajectory-tracking control law using static state feedback linearization [20] is implemented. This control law guarantees, for any kinematic configuration, a second order error dynamics by the selection of a specific virtual reference point located on the cart (with some constraints). For a differential drive robot, the virtual point could be located anywhere on the chassis except on the line (virtual axis) joining the driving wheels.

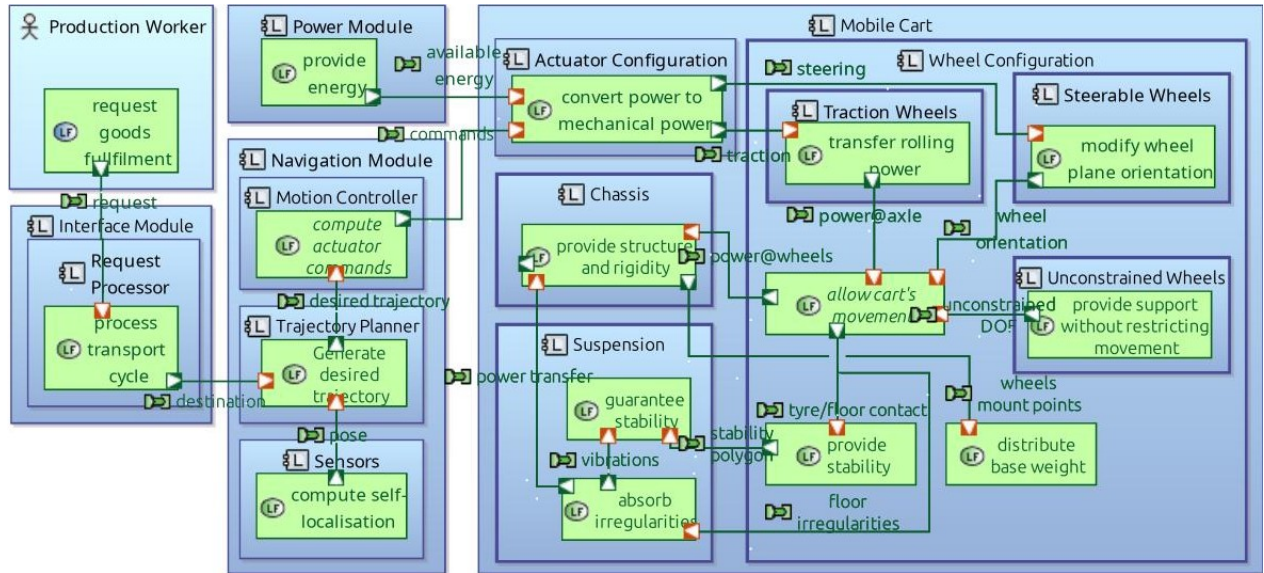


Figure 3: Logical Architecture Blank: Mobile Base

3.4 Physical Architecture

In Figure 4 the initial Physical Architecture for a 4-wheel differential drive (two actuated rear wheels and two caster front wheels) is shown. Physical nodes are represented by *Node Components* (yellow blocks) (e.g. *Chassis*, *Wheels*, *Actuators*) that provide the necessary material resources to deploy *Behaviour Components* (in blue) (e.g. *RigidBody*, *FixedWheel*, *CasterWheel*, *ElectricalMotor*) that, in turn, characterize the behaviour and perform functions (green blocks). In this particular view of the system, *Functional Exchanges* represent power flow between components. The exchanges between behaviour components, called *Component Exchanges*, specify conditions for the functional exchanges (e.g. *revoluteJoint*, *fixedJoint*, *transmission chains*). The node components are connected via *Physical Links* which represent how the nodes physically connects to each others (e.g. *bearing assemblies*, *mounting plates*, *transmission assembly*).

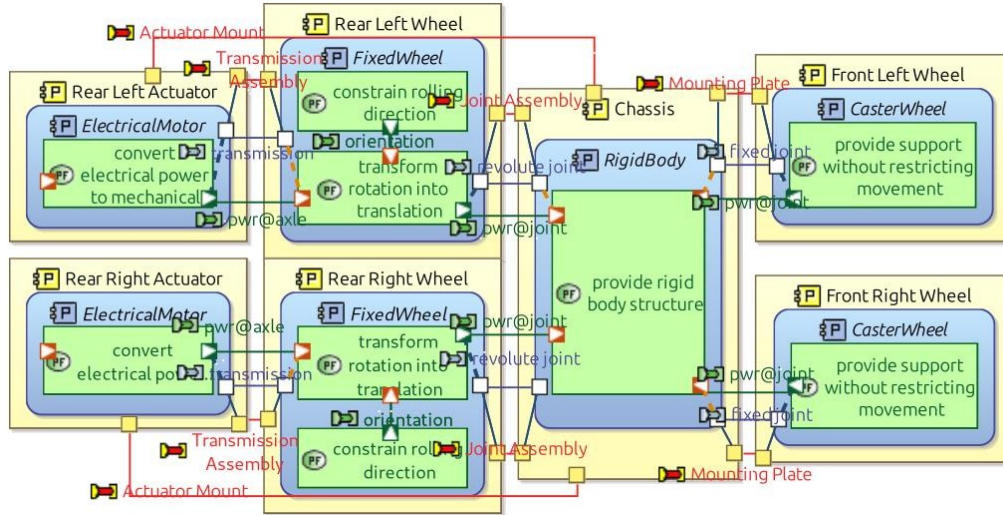


Figure 4: Physical Architecture Blank: Differential Drive Mobile Base

The mobile cart's physical architecture can be mapped to simulation models to verify the overall system behaviour and also to acquire values for relevant properties that drive the detailed design processes. Multidomain physical models are required to capture different level of details, complexity and physical phenomena in the simulations as long as the design matures. A natural tool chosen for these expandable simulation models are the Bond Graphs (BG) [21] where components exchange power through their interfaces and additional physical phenomena (e.g. inertia, friction) could be easily added or removed in the specific implementation of each of them. In Figure 5a a very high-level Word-BG model is shown, directly mapping the physical architecture of the cart connected to the controller's actuation signals. A schematic of the cart, reference path and the virtual point used for the controller are illustrated in Figure 5b.

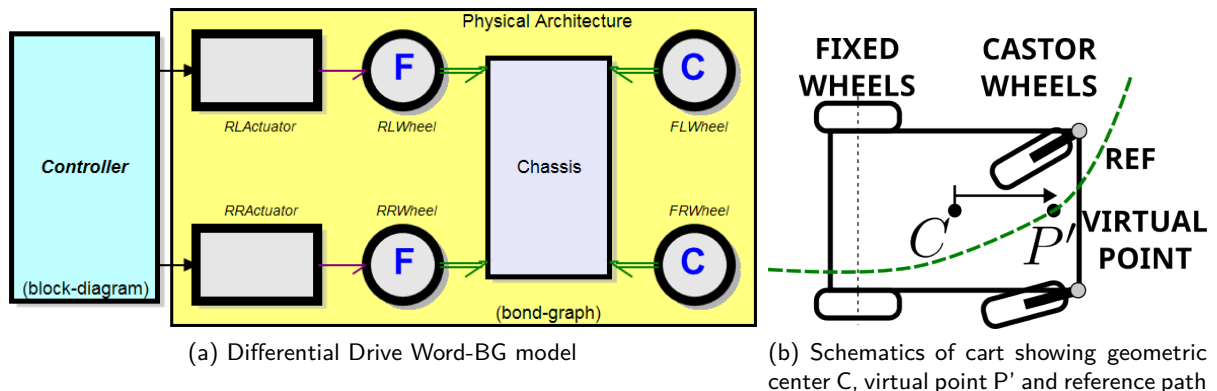


Figure 5: Controlled Word-BG simulation model and illustration of the virtual reference point on the cart

4. DETAILED DESIGN

The input parameters needed for the detailed design of each component are roughly estimated based on functions that are traced to the system architecture or requirements refined iteratively throughout the design process.

To begin with, to estimate the rough dimensions of the cart (length L and width W), we can trace back to the stakeholder requirement that specifies the maximum load size, which constrains the cart's dimension to be at least $50 \times 50\text{cm}$, and to the requirement to stay inside the pathways painted on the floor, which is refined by measuring the pathways width A , and directly provides cart's width upper-bound of $W < A$. A cornering study provides the maximum length L for which a rectangle of width W geometrically fits in the pathways, by making sure that the distance to the vertex of the corner is $d > 0$ at all cart-angles. The cornering model and relation between L and W are shown in Figure 6a where the condition $A = 80\text{cm}$ is given. The shadowed area represents all the valid combinations of W and L : for thinner carts ($W \approx 55\text{cm}$) max length is around $L \leq 110\text{cm}$ whereas for wider carts ($W \approx 75\text{cm}$), max length is around $L \leq 80\text{cm}$.

In second place, to succesfully overcome the ramps of angle θ identified in the factory, a study is designed to obtain the relation between the floor-clearance of the cart h and the wheelbase B as shown in Figure 6b. The shadowed area represents the valid combinations of h and B constraining maximum ramp angles at $\theta = 15^\circ$. For carts with wheelbase of around $B \approx 90\text{cm}$, cart's clearance should be at least $h \geq 6.5\text{cm}$.

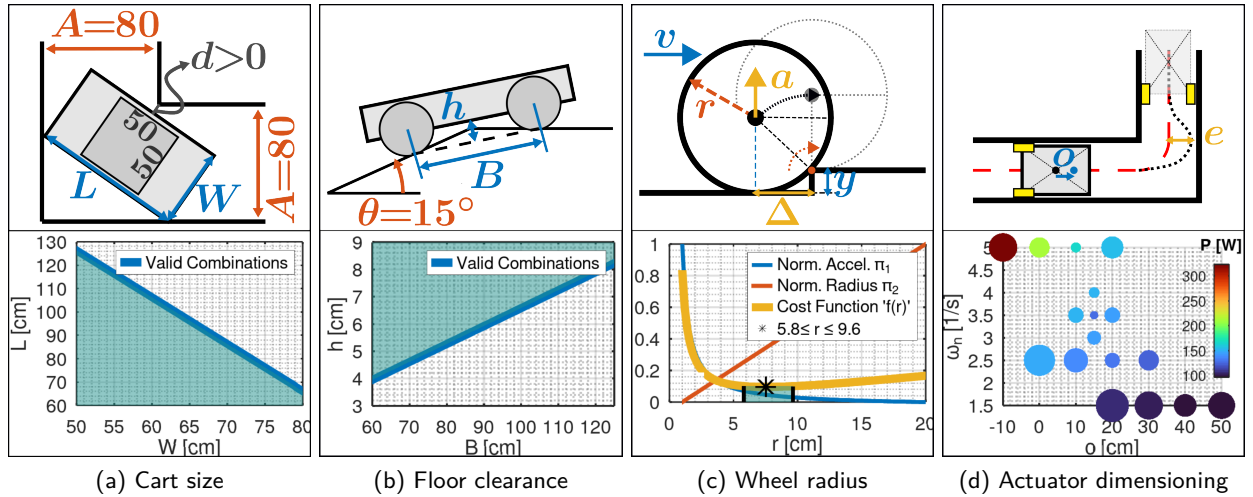


Figure 6: Design functions for estimating initial rough dimensions.

Additionally, the wheel radius shall be large enough to overcome the irregularities on the floor while being as small as possible to optimize geometrical space. In the top image of Figure 6c, a model to estimate the vertical accelerations a is shown where irregularities are modeled as steps of height y . The vertical accelerations are estimated as $a \sim y \cdot v^2 / \Delta^2$ where v is the cart's speed and Δ is the contact points distance at the impact moment (bigger as r increases). Two adimensional parametres ($\pi_1 = a \cdot y / v^2$) and ($\pi_2 = r / y$) are found, isolating the minimal acceleration problem from step height and velocity [22]. A cost function f that minimizes both π_1 and π_2 is constructed by assigning weight coefficients and normalizing each of them to $\pi_{1,2} \in [0, 1]$ intervals.



IX CAIM
IV CAIFE



FIE
Facultad de
Ingeniería
del Ejército



FoDAMI

17, 18 y 19 de septiembre 2025

Minimizing the accelerations is considered more important than minimizing the geometrical space required for the wheels (to a certain extent), assigning a weight ratio of $w_1/w_2 = 5$ to the cost function. In the bottom image of the figure, the adimensional parameters and the cost function are plotted against r (where $y \sim 1\text{cm}$ is considered). As a result, valid radius fall in the interval $5.8\text{cm} \leq r \leq 9.6\text{cm}$ which minimizes the cost function within 5%.

Finally, as the last design function presented here, a simulated kinematic cart model with derivative inertias is used to verify the controller and to estimate approximate power ratings for the motors in ideal working situations. The scenario shown here is one of the most power demanding scenario for the actuators: rotating 90-degrees at a corner at constant speed (1m/s) without deviating from the pathways while loading maximal weight (150kg). Controller's second order gains were iterated by fixing maximally flat damping value at $\xi = \sqrt{2}/2$ and varying frequency ω_n . The virtual following point offset from the geometric center of the cart o is also varied to verify its impact on the results. Notice that the error path computed is not the error of the virtual point, but rather the deviation from the geometric center of the cart to the pathway center reference path, which will better represent the total cart's deviation. The results are shown as a bubble chart in Figure 6d, where x axis represents the virtual point's offset o , y axis is the controller's frequency ω_n , the size of the bubbles is the resulting path error e (the smaller the better) and the colormap represent the maximum power output required by each of the actuators P . As a result, simulations suggest that the controller's following point shall be a little ahead of the geometric center of the cart (around 15cm) and 150W actuators should be a good starting estimate.

5. CONCLUSIONS

In this paper, the ongoing state of development of a mobile robot for material transportation in a metalworking factory is presented. We follow the V-model for mechatronic design by making use of the Arcadia MBSE method and language and the Capella tool. The initial analysis and modeling results of the main four perspectives of Arcadia are detailed, obtaining a first version of the architecture of the mobile robot. Lastly, a rough estimate of parameters that will drive the first iteration of the detailed design of the components are obtained by tracing the physical components and design functions to the system requirements.

For future work, our main objective is to obtain an initial virtual primitive version of the robot, by completing a full iteration of the V-model. For this, initial low-detail domain-specific design activities will be conducted and IVV test scenarios will be defined: scenarios for verifying tip-over stability of the mobile base while turning, and/or scenarios for verifying the controller's capability to prevent accidents, for example.

6. REFERENCES

- [1] Zheng, C., Bricogne, M., Le Duigou, J., & Eynard, B. (2014). Survey on mechatronic engineering: A focus on design methods and product models. *Advanced Engineering Informatics*, 28(3), 241–257.
- [2] Barbieri, G., Fantuzzi, C., & Borsari, R. (2014). A model-based design methodology for the development of mechatronic systems. *Mechatronics*, 24(7), 833–843. <https://doi.org/10.1016/j.mechatronics.2013.12.004>
- [3] Madni, A. M., & Sievers, M. (2018). Model-based systems engineering: Motivation, current status, and research opportunities. *Systems Engineering*, 21(3), 172–190. <https://doi.org/10.1002/sys.21438>



**IX CAIM
IV CAIFE**



FIE
Facultad de
Ingeniería
del Ejército



FoDAMI

17, 18 y 19 de septiembre 2025

-
- [4] Estefan, J. A. (2007). Survey of model-based systems engineering (MBSE) methodologies. *In cose MBSE Focus Group*, 25(8), 1–12.
 - [5] Voirin, J.-L. (2018). *Model-based system and architecture engineering with the arcadia method*. Elsevier.
 - [6] Aleksandraviciene, A., & Morkevicius, A. (2018). *Magicgrid® book of knowledge-a practical guide to systems modeling using magicgrid from no magic*. Inc, Allen Texas, USA.
 - [7] Hernandez, C., & Fernandez-Sanchez, J. L. (2017). Model-based systems engineering to design collaborative robotics applications. *2017 IEEE ISSE*, 1–6. <https://doi.org/10.1109/SysEng.2017.8088258>
 - [8] Berschik, M. C., Schumacher, T., Laukotka, F. N., Krause, D., & Inkermann, D. (2023). MBSE within the engineering design community—an exploratory study. *Proceedings of the Design Society*, 3, 2595–2604.
 - [9] Bouhali, I., Pasquariello, A., Mhenni, F., Vitolo, F., Hehenberger, P., Patalano, S., & Choley, J.-Y. (2025). Model-based systems engineering and safety assessment: A workflow for mechatronic systems design. *Systems Engineering*, 28(2), 238–254. <https://doi.org/10.1002/sys.21791>
 - [10] Graessler, I., Wiechel, D., & Oleff, C. (2022). Extended RFLP for complex technical systems. *2022 IEEE ISSE*, 1–8. <https://doi.org/10.1109/ISSE54508.2022.10005424>
 - [11] Graessler, I., & Hentze, J. (2020). The new v-model of VDI 2206 and its validation. *at - Automatisierungstechnik*, 68(5), 312–324. <https://doi.org/10.1515/auto-2020-0015>
 - [12] Vitolo, F., Rega, A., Di Marino, C., Pasquariello, A., Zanella, A., & Patalano, S. (2022). Mobile robots and cobots integration: A preliminary design of a mechatronic interface by using MBSE approach. *Applied Sciences*, 12(1), 419. <https://doi.org/10.3390/app12010419>
 - [13] Younse, P. J., Cameron, J. E., & Bradley, T. H. (2021). Comparative analysis of model-based and traditional systems engineering approaches for architecting a robotic space system through automatic information transfer. *IEEE Access*, 9, 107476–107492. <https://doi.org/10.1109/ACCESS.2021.3096468>
 - [14] Portillo-Vélez, R. D. J., Burgos-Castro, I. A., Vásquez-Santacruz, J. A., & Marín-Urías, L. F. (2022). Integrated conceptual mechatronic design of a delta robot. *Machines*, 10(3), 186.
 - [15] Zhang, J., Wang, R., Liu, T., Zhang, J., Zhang, S., & Zhang, H. (2023). Design and modeling of nuclear power inspection robot based on MBSE. In *International Conference "Complex Systems Design & Management"* (pp. 289–300, Vol. 1085). Springer Nature Singapore.
 - [16] Miyoshi, T., & Kimura, T. (2021). A reference model in safety design of a modular service robot based on MBSE: A case study of a stocking robot in a supermarket. *2021 IEEE ISR*, 399–402.
 - [17] Roques, P. (2016). MBSE with the ARCADIA method and the capella tool. *ERTS 2016*.
 - [18] INCOSE (Ed.). (2023). *Systems engineering handbook* (Fifth edition). John Wiley & Sons.
 - [19] Comerford, R. (1994). Mecha...what? [mechatronics]. *IEEE Spectrum*, 31(8), 46–49.
 - [20] Canudas de Wit, C., Siciliano, B., & Bastin, G. (Eds.). (1996). *Theory of robot control*. Springer London. <https://doi.org/10.1007/978-1-4471-1501-4>
 - [21] Karnopp, D. C., Margolis, D. L., & Rosenberg, R. C. (2012, February 8). *System dynamics: Modeling, simulation, and control of mechatronic systems* (1st ed.). Wiley. <https://doi.org/10.1002/9781118152812>
 - [22] Lemons, D. S. (2017). *A student's guide to dimensional analysis*. Cambridge University Press.